LAPORAN PRAKTIKUM

Modul 10 - DBMS dan Aplikasi Basis Data



Disusun oleh:

BRAMANTA SETYA BAGASKARA	**264311009**
EDGAR RIZKI PUTRA NOVARIYANTO	**264311013**
RIFQI ADYASTA MAHARDIKA	**264311029**

JURUSAN TEKNIK
TEKNOLOGI REKAYASA PERANGKAT LUNAK
POLITEKNIK NEGERI MADIUN
2026

1. Identifikasi DBMS

1.1 Hasil Identifikasi

Komponen	Hasil Identifikasi
DBMS	MySQL/MariaDB
Versi DBMS	MySQL 8.4.3 / MariaDB 11.8.8
Vendor	MySQL Community Server - GPL
Storage engine yang digunakan	InnoDB
Aplikasi pengelola basis data	DBeaver, phpMyAdmin, dan MySQL/MariaDB CLI
Sistem operasi	Windows 10/11

Tabel 1. Identifikasi lingkungan DBMS

1.2 Tangkapan Layar

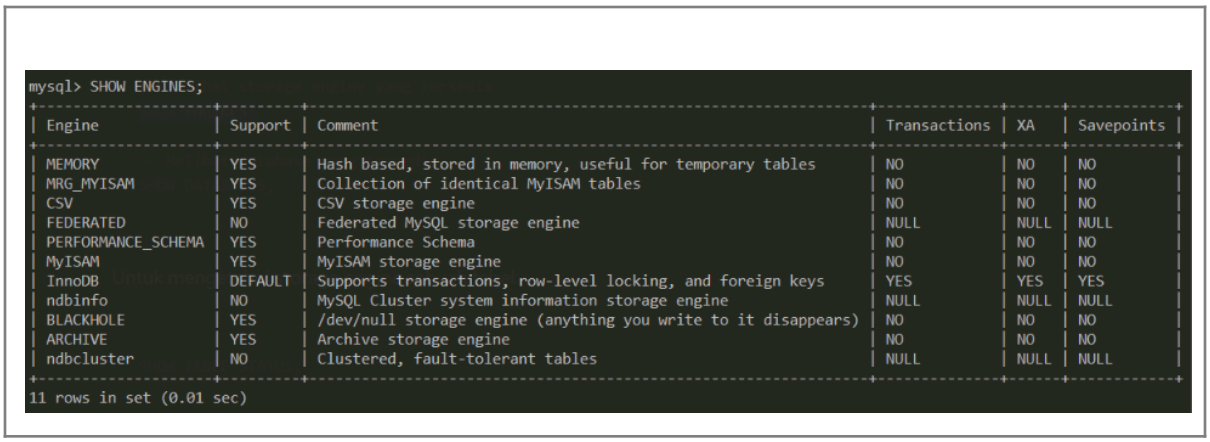
Versi *DBMS:*

```
mysql> SELECT VERSION();
+-----------+
| VERSION() |
+-----------+
| 8.4.3     |
+-----------+
1 row in set (0.00 sec)
```

Vendor:

```
mysql> SELECT @@version_comment;
+------------------------------+
| @@version_comment            |
+------------------------------+
| MySQL Community Server - GPL |
+------------------------------+
1 row in set (0.00 sec)
```

Engine:



```
mysql> SHOW ENGINES;
+--------------------+---------+----------------------------------------------------------------+--------------+------+------------+
| Engine             | Support | Comment                                                        | Transactions | XA   | Savepoints |
+--------------------+---------+----------------------------------------------------------------+--------------+------+------------+
| MEMORY             | YES     | Hash based, stored in memory, useful for temporary tables      | NO           | NO   | NO         |
| MRG_MYISAM         | YES     | Collection of identical MyISAM tables                          | NO           | NO   | NO         |
| CSV                | YES     | CSV storage engine                                             | NO           | NO   | NO         |
| FEDERATED          | NO      | Federated MySQL storage engine                                 | NULL         | NULL | NULL       |
| PERFORMANCE_SCHEMA | YES     | Performance Schema                                             | NO           | NO   | NO         |
| MyISAM             | YES     | MyISAM storage engine                                          | NO           | NO   | NO         |
| InnoDB             | DEFAULT | Supports transactions, row-level locking, and foreign keys     | YES          | YES  | YES        |
| ndbinfo            | NO      | MySQL Cluster system information storage engine                | NULL         | NULL | NULL       |
| BLACKHOLE          | YES     | /dev/null storage engine (anything you write to it disappears) | NO           | NO   | NO         |
| ARCHIVE            | YES     | Archive storage engine                                         | NO           | NO   | NO         |
| ndbcluster         | NO      | Clustered, fault-tolerant tables                               | NULL         | NULL | NULL       |
+--------------------+---------+----------------------------------------------------------------+--------------+------+------------+
11 rows in set (0.01 sec)
```

Gambar 1. Hasil identifikasi DBMS

1.3 Matriks Penilaian Perangkat Lunak Pengelola

Pertimbangan	phpMyAdmin	DBeaver	MySQL/Maria DB CLI
Kompatibel dengan DBMS dan sistem operasi	Sangat kompatibel di berbagai perangkat selama dapat mengakses web	Kompatibel dengan Windows, MacOS dan Linux distro	Sangat kompatibel di berbagai perangkat selama memiliki akses ke terminal
Mudah digunakan untuk administrasi	Mudah	Sedang	Sulit
Mendukung editor dan eksekusi query	Mendukung	Mendukung	Mendukung
Mendukung import, export, dan pengelolaan tabel	Mendukung	Mendukung	Mendukung
Mendukung autentikasi dan hak akses	Mendukung	Mendukung	Mendukung
Mendukung pemantauan dan analisis query	Mendukung	Mendukung	Mendukung
Mendukung backup, restore, dan pengelolaan skema	Mendukung	Mendukung	Mendukung
Lisensi sesuai kebutuhan dan anggaran	GPL (gratis dan open source)	Community (gratis) / Pro (berbayar)	GPL (gratis dan open source)

TOTAL SKOR	9/10	9.2/10	8/10

Tabel 2. Matriks penilaian aplikasi pengelola basis data

1.4 Alasan Pemilihan Perangkat Lunak

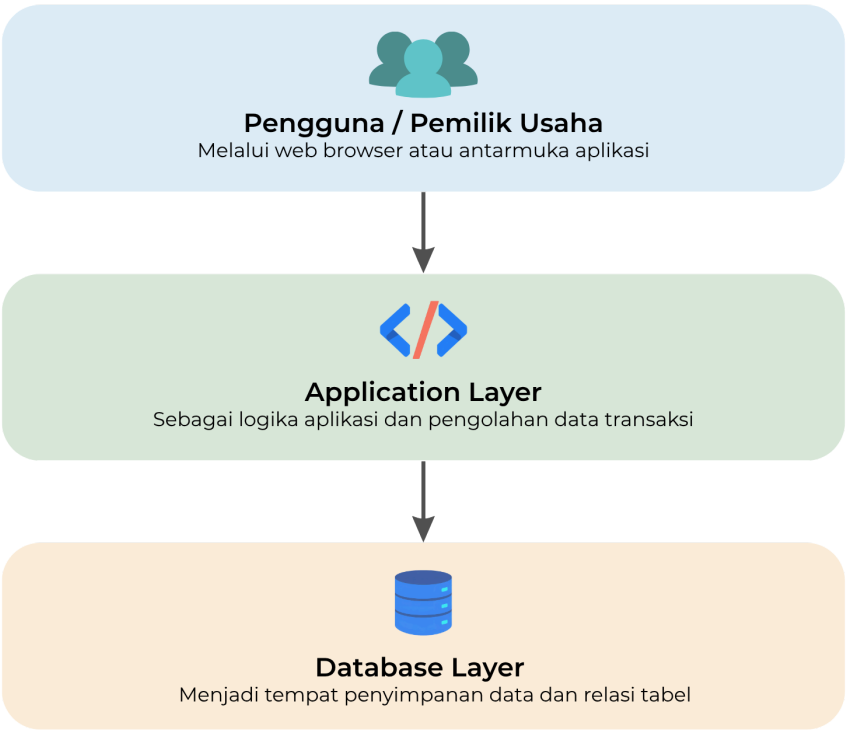
Perangkat lunak manajemen basis data yang dipilih untuk proyek laporan keuangan UMKM berbasis MySQL adalah DBeaver Community Edition. Pemilihan ini didasarkan pada kelengkapan fitur yang sesuai dengan kebutuhan proyek. DBeaver mendukung MySQL sekaligus berbagai DBMS lain dan dapat berjalan di Windows, macOS, maupun Linux, sehingga memberikan fleksibilitas apabila proyek dikembangkan di kemudian hari. DBeaver juga menyediakan editor query dengan fitur auto-complete dan visualisasi execution plan yang memudahkan tim developer menyusun query kompleks, seperti rekap transaksi dan perhitungan laporan keuangan. Selain itu, DBeaver mendukung import, export, serta backup dan restore skema, sehingga kebutuhan backup rutin dan pengarsipan data keuangan dapat terpenuhi. Pengelolaan autentikasi dan hak akses juga tersedia, sehingga akses basis data dapat dibatasi hanya kepada tim developer. Sebagai perangkat lunak open source, DBeaver tidak memerlukan biaya lisensi dan sesuai dengan anggaran proyek.

Meskipun memiliki banyak kelebihan, DBeaver juga memiliki beberapa keterbatasan. Antarmuka DBeaver lebih kompleks dibandingkan tools berbasis web seperti phpMyAdmin, sehingga membutuhkan penyesuaian bagi pengguna baru. DBeaver juga merupakan aplikasi desktop sehingga tidak dapat diakses langsung melalui browser. Selain itu, sebagian fitur analitik lanjutan hanya tersedia pada versi Pro yang berbayar. Namun, keterbatasan tersebut tidak menjadi kendala berarti dalam proyek ini. Pengguna basis data hanya tim developer yang memiliki kompetensi teknis memadai, sementara pengguna akhir aplikasi mengakses sistem melalui browser dan tidak terhubung langsung ke basis data. Kebutuhan utama proyek, yaitu pengelolaan data, query rekap transaksi, dan backup rutin, juga sudah tercakup dalam versi Community yang gratis.

Dengan demikian, DBeaver dinilai paling sesuai untuk proyek laporan keuangan UMKM ini. DBeaver menawarkan kelengkapan fitur teknis, dukungan backup dan pengarsipan data, pengelolaan hak akses, serta lisensi gratis yang selaras dengan kebutuhan dan anggaran proyek. Keterbatasan yang dimiliki tidak berpengaruh signifikan karena pengguna basis data terbatas pada tim developer. Oleh karena itu, DBeaver menjadi pilihan yang tepat dan mendukung kelancaran pengelolaan basis data MySQL pada aplikasi laporan keuangan UMKM.

2. Arsitektur Sistem

Berikut merupakan diagram arsitektur sistem dengan tipe three-tier:



Gambar 2. Diagram arsitektur implementasi sistem

2.1 Penjelasan Arsitektur

Lapisan / Komponen	Teknologi yang Digunakan	Fungsi
Presentation Layer / Front-end	Laravel Blade	Menampilkan antarmuka aplikasi kepada pengguna (form input transaksi, dashboard laporan keuangan, tabel rekap) dan mengirim data ke application layer.
Application Layer / Back-end	Laravel (PHP Framework)	Memproses logika bisnis: validasi input, perhitungan laba-rugi, pengelolaan transaksi, serta menghubungkan

		presentation layer dengan database.
Database Layer	MySQL atau MariaDB	Menyimpan seluruh data keuangan (transaksi, akun, laporan), mendukung query rekap transaksi, serta backup dan restore data.

Tabel 3. Komponen arsitektur sistem

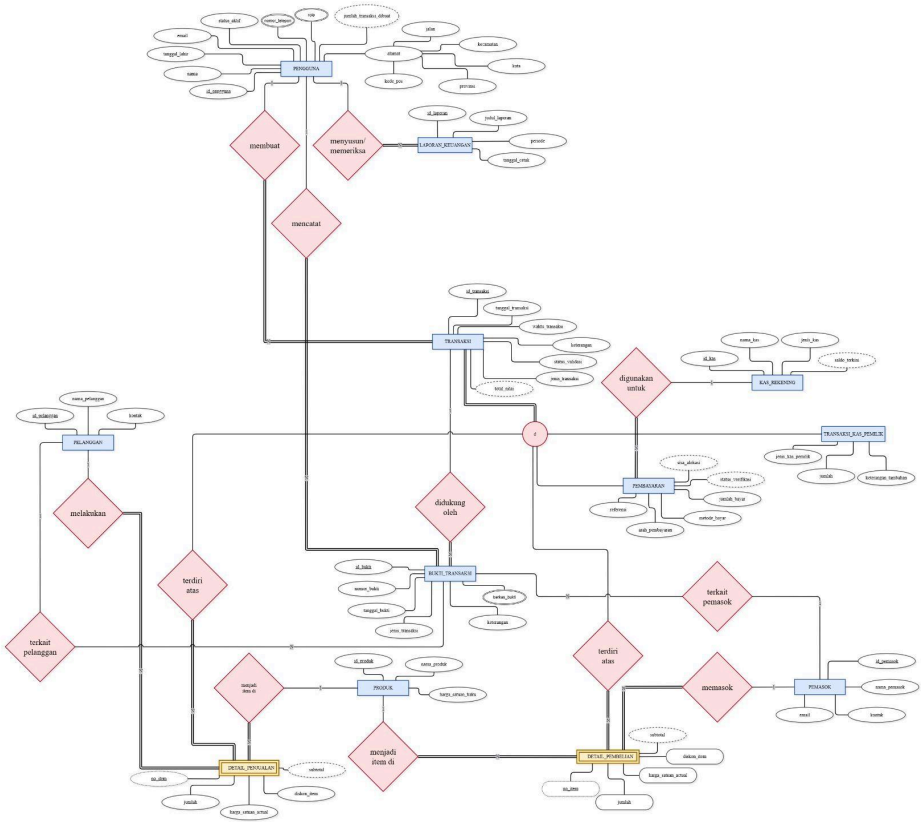
Sistem laporan keuangan UMKM ini menggunakan arsitektur client-server berbasis web dengan pola MVC (Model-View-Controller) yang merupakan pola bawaan framework Laravel. Arsitektur ini membagi sistem menjadi tiga lapisan, yaitu presentation layer, application layer, dan database layer, sehingga setiap komponen memiliki tanggung jawab yang jelas dan terpisah.

Presentation layer dibangun menggunakan Laravel Blade yang menghasilkan tampilan HTML dan diakses pengguna melalui browser. Lapisan ini berfokus pada penyajian antarmuka, seperti form input transaksi, dashboard laporan, dan tabel rekap, tanpa menangani logika bisnis. Application layer menggunakan Laravel sebagai kerangka kerja utama yang memproses seluruh logika bisnis, meliputi validasi input, perhitungan laba-rugi, pengelolaan transaksi, serta pengaturan hak akses. Lapisan ini juga menjadi penghubung antara tampilan dan basis data melalui Eloquent ORM dan query builder. Database layer menggunakan MySQL untuk menyimpan seluruh data keuangan secara terstruktur dan mendukung kebutuhan query rekap transaksi serta backup dan restore data.

Pemilihan arsitektur ini didasarkan pada beberapa alasan. Pertama, pemisahan tanggung jawab (separation of concerns) membuat kode lebih terstruktur, mudah dipelihara, dan mudah dikembangkan. Kedua, pola MVC yang merupakan bawaan Laravel mempercepat proses pengembangan karena menyediakan struktur direktori, routing, validasi, dan keamanan dasar (seperti proteksi CSRF dan hashing password) yang siap digunakan. Ketiga, arsitektur berbasis web memungkinkan aplikasi diakses melalui browser tanpa instalasi di sisi pengguna, sehingga sesuai untuk UMKM yang membutuhkan akses laporan secara praktis. Keempat, penggunaan MySQL sebagai basis data relasional sesuai untuk data keuangan yang bersifat terstruktur dan saling berelasi (transaksi, akun, dan laporan), serta mendukung kebutuhan backup rutin dan query agregasi untuk rekap laporan.

3. Implementasi SQL

3.1 Rancangan Skema



Gambar 3. ERD basis data

3.2 Script DDL

https://github.com/Aaksaa/DB-KELOMPOK-8/blob/main/Modul%2010/3-Implementasi%20SQL/1-create-table.sql

3.3 Script DML (Data Awal)

Operasi DML(memasukkan data):
https://github.com/Aaksaa/DB-KELOMPOK-8/blob/main/Modul%2010/3-Implementasi%20SQL/2-create.sql
Read:
https://github.com/Aaksaa/DB-KELOMPOK-8/blob/main/Modul%2010/3-Implementasi%20SQL/3-read.sql
Update:
https://github.com/Aaksaa/DB-KELOMPOK-8/blob/main/Modul%2010/3-Implementasi%20SQL/4-update.sql
Delete:
https://github.com/Aaksaa/DB-KELOMPOK-8/blob/main/Modul%2010/3-Implementasi%20SQL/5-delete.sql

3.4 Ringkasan Struktur Tabel

Tabel	Primary Key	Foreign Key	Constraint Lain
PENGGUNA	`id_pengguna` (AUTO_INCREMENT)	-	• `nama`, `tanggal_lahir`, `email`, `status_aktif`: NOT NULL • `alamat`: VARCHAR(255) - composite attribute • `nomor_telepon`: TEXT - multivalued • `role`: TEXT - multivalued • `jumlah_transaksi_dibuat`: INT - derived attribute
PRODUK	`id_produk` (AUTO_INCREMENT)	-	• `nama_produk`: NOT NULL

			• harga_satuan_baku: DECIMAL(15,2) NOT NULL
PELANGGAN	id_pelanggan (AUTO_INCREMENT)	-	• nama_pelanggan: NOT NULL
PEMASOK	id_pemasok (AUTO_INCREMENT)	-	• nama_pemasok: NOT NULL
KAS_REKENING	id_kas (AUTO_INCREMENT)	-	• nama_kas, jenis_kas: NOT NULL • saldo_terkini: DECIMAL(15,2) - derived attribute
LAPORAN_KEUANGAN	id_laporan (AUTO_INCREMENT)	id_pengguna → PENGGUNA(id_pengguna) ON UPDATE CASCADE ON DELETE RESTRICT	• judul_laporan, periode, tanggal_cetak: NOT NULL • id_pengguna: NOT NULL
TRANSAKSI	id_transaksi (AUTO_INCREMENT)	id_pengguna → PENGGUNA(id_pengguna) ON UPDATE CASCADE ON DELETE RESTRICT	• tanggal_transaksi, waktu_transaksi, status_validasi, jenis_transaksi: NOT NULL • total_nilai: DECIMAL(15,2) - derived attribute • jenis_transaksi:

			discriminator untuk inheritance
BUKTI_TRANSAKSI	id_bukti (AUTO_INCREMENT)	• id_transaksi → TRANSAKSI(id_transaksi) ON UPDATE CASCADE ON DELETE RESTRICT • id_pelanggan → PELANGGAN(id_pelanggan) ON UPDATE CASCADE ON DELETE RESTRICT • id_pemasok → PEMASOK(id_pemasok) ON UPDATE CASCADE ON DELETE RESTRICT • id_pengguna → PENGGUNA(id_pengguna) ON UPDATE CASCADE ON DELETE RESTRICT	• nomor_bukti, tanggal_bukti, jenis_transaksi, id_pengguna: NOT NULL • berkas_bukti: TEXT - multivalued • id_pelanggan, id_pemasok: NULL (opsional)
PEMBAYARAN	id_transaksi	• id_transaksi → TRANSAKSI(id_transaksi) ON UPDATE CASCADE ON DELETE CASCADE • id_kas → KAS_REKENING(id_kas)	• arah_pembayaran, metode_bayar, jumlah_bayar, id_kas: NOT NULL • status_verifikasi: derived attribute • sisa_alokasi: derived attribute

		ON UPDATE CASCADE ON DELETE RESTRICT	
TRANSAKSI_KAS_PEMILIK	`id_transaksi`	`id_transaksi` → TRANSAKSI(id_transaksi) ON UPDATE CASCADE ON DELETE CASCADE	• `jenis_kas_pemilik`, `jumlah`: NOT NULL
DETAIL_PENJUALAN	Composite: `(id_transaksi, no_item)`	• `id_transaksi` → TRANSAKSI(id_transaksi) ON UPDATE CASCADE ON DELETE CASCADE • `id_produk` → PRODUK(id_produk) ON UPDATE CASCADE ON DELETE RESTRICT • `id_pelanggan` → PELANGGAN(id_pelanggan) ON UPDATE CASCADE ON DELETE RESTRICT	• `no_item`: partial key • `id_produk`, `id_pelanggan`, `jumlah`, `harga_satuan_actual`, `diskon_item`: NOT NULL • `subtotal`: derived attribute
DETAIL_PEMBELIAN	Composite: `(id_transaksi, no_item)`	• `id_transaksi` → TRANSAKSI(id_transaksi) ON UPDATE CASCADE ON DELETE CASCADE • `id_produk` → PRODUK(id_produk) ON UPDATE CASCADE ON DELETE RESTRICT	• `no_item`: partial key • `id_produk`, `id_pemasok`, `jumlah`, `harga_satuan_actual`, `diskon_item`: NOT NULL • `subtotal`: derived attribute

		• `id_pemasok` → PEMASOK(id_pemasok) ON UPDATE CASCADE ON DELETE RESTRICT	

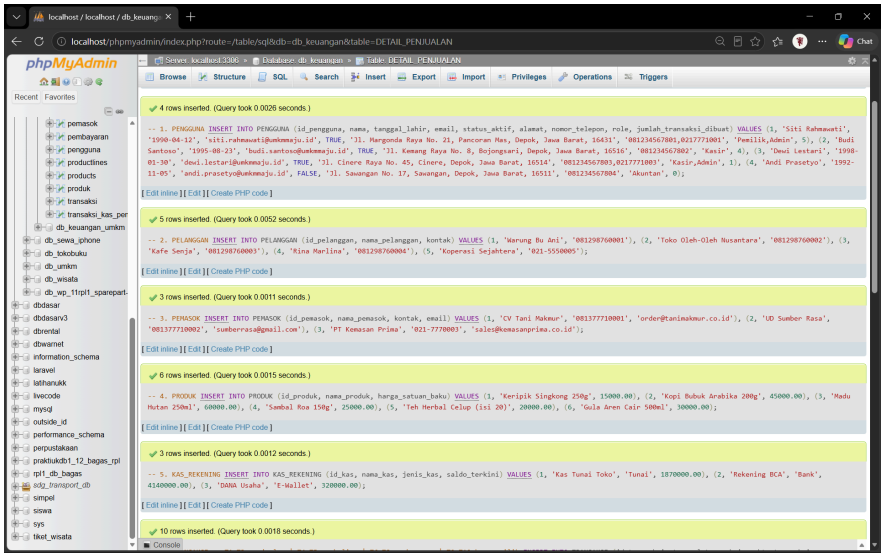
Tabel 4. Primary key, foreign key, dan constraint

3.5 Hasil Pengujian CRUD

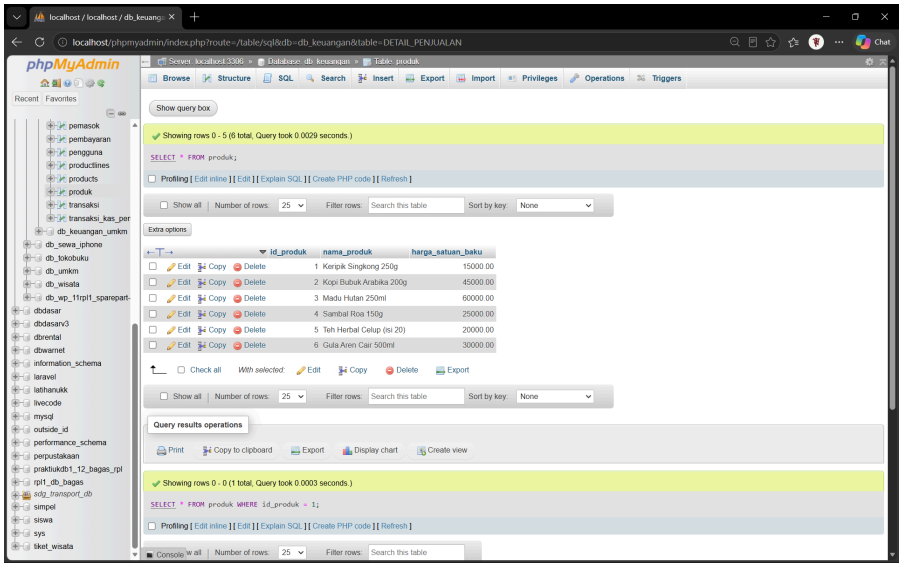
Operasi	Fungsinya
CREATE (INSERT)	Menambahkan data
READ (SELECT)	Melihat data
UPDATE	Mengubah data
DELETE	Menghapus data

Tabel 5. Hasil pengujian operasi CRUD

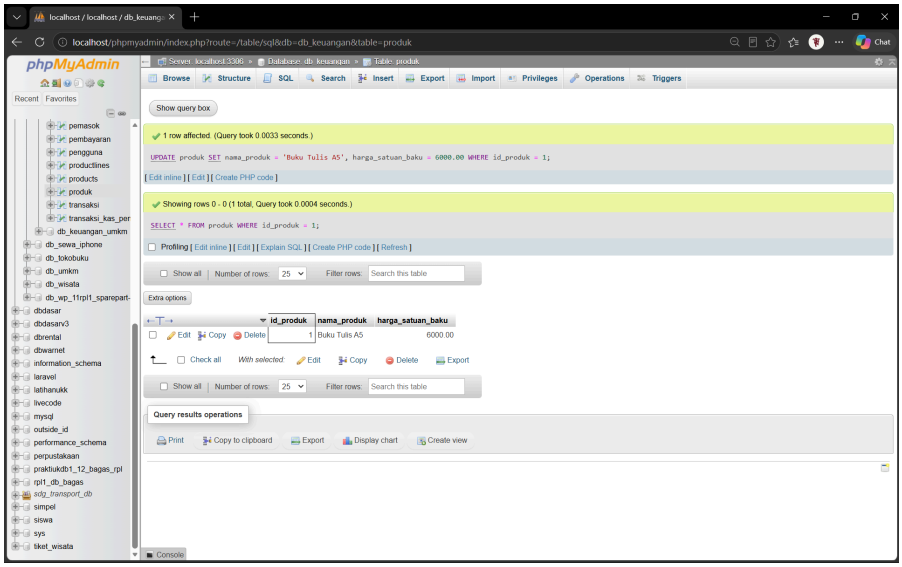
CREATE :



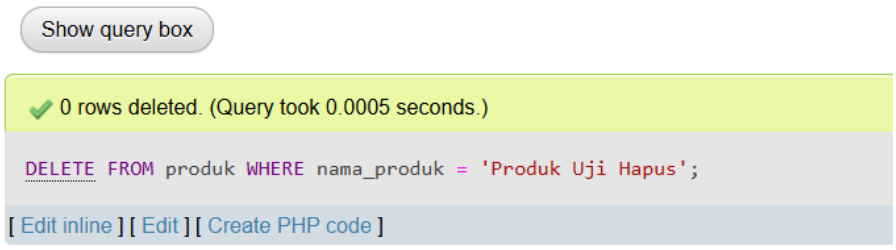
READ :



UPDATE :



DELETE :



Gambar 4. Bukti pengujian CRUD

4. Pengujian Integritas

4.1 Uji Integritas Referensial

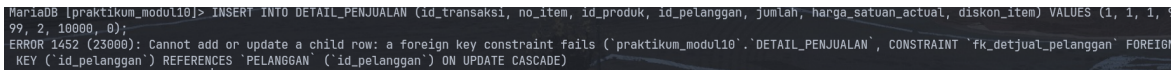
Skenario Uji	Query	Hasil yang Diharapkan	Hasil Aktual (pesan error)
INSERT detail penjualan dengan id_pelanggan tidak terdaftar (999)	INSERT INTO DETAIL_PENJUALAN (id_transaksi, no_item, id_produk, id_pelanggan, jumlah, harga_satuan_actual, diskon_item) VALUES (1, 1, 1, 999, 2, 10000, 0);	GAGAL (FK fk_detjual_pelanggan)	ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`praktikum_modul10`.`DETAIL_PENJUALAN`, CONSTRAINT `fk_detjual_pelanggan` FOREIGN KEY (`id_pelanggan`) REFERENCES `PELANGGAN` (`id_pelanggan`) ON UPDATE CASCADE)
DELETE pelanggan yang masih memiliki pesanan	DELETE FROM PELANGGAN WHERE id_pelanggan = 1;	GAGAL (RESTRICT default, FK tidak mendefinisikan ON DELETE)	ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`praktikum_modul10`.`DETAIL_PENJUALAN`, CONSTRAINT `fk_detjual_pelanggan` FOREIGN KEY (`id_pelanggan`) REFERENCES `PELANGGAN` (`id_pelanggan`) ON

			UPDATE CASCADE)
Uji constraint lain (Foreign key)	INSERT INTO PEMBAYARAN (id_transaksi, arah_pembayaran, metode_bayar, jumlah_bayar, id_kas) VALUES (2, 'masuk', 'transfer', 50000, 999);	GAGAL (FK fk_pembayaran_kas)	ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`praktikum_modul10`.`PEMBAYARAN`, CONSTRAINT `fk_pembayaran_kas` FOREIGN KEY (`id_kas`) REFERENCES `KAS_REKENING` (`id_kas`) ON UPDATE CASCADE)

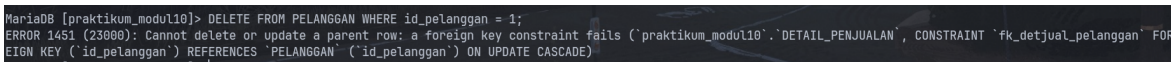
Tabel 6. Hasil pengujian integritas

Berikut *screenshot* pesan *error foreign key / constraint*

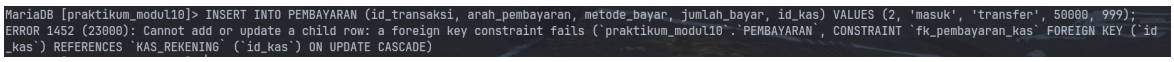
1.



2.



3.



Gambar 5. Bukti pengujian integritas

4.2 Analisis

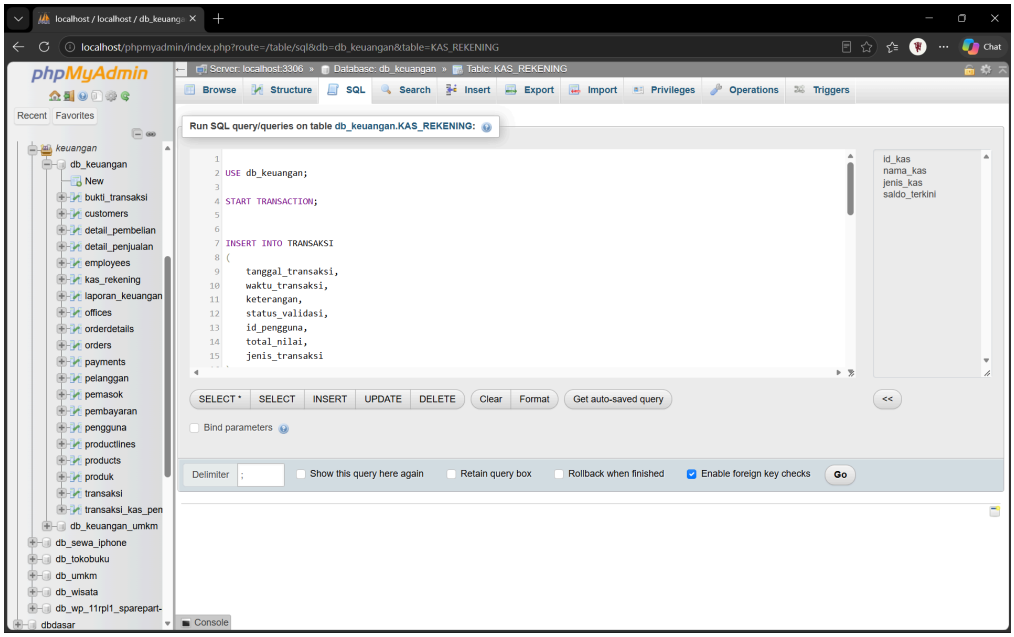
Primary Key (PK) merupakan kolom atau kombinasi kolom yang secara unik mengidentifikasi setiap baris dalam tabel. Karena bersifat unik dan tidak boleh kosong, PK menjamin integritas entitas (entity integrity), yaitu memastikan tidak terdapat data ganda dan setiap baris memiliki identitas yang jelas. Sebagai contoh, PENGGUNA.id_pengguna sebagai PK mencegah adanya dua pengguna dengan ID yang sama, sementara DETAIL_PENJUALAN menggunakan PK komposit (id_transaksi, no_item) agar setiap item dalam satu transaksi bersifat unik. Penerapan PK ini terbukti ketika dilakukan penyisipan data dengan ID yang telah ada, sistem menolak dengan pesan ERROR 1062 (23000): Duplicate entry '1' for key 'PRIMARY'.

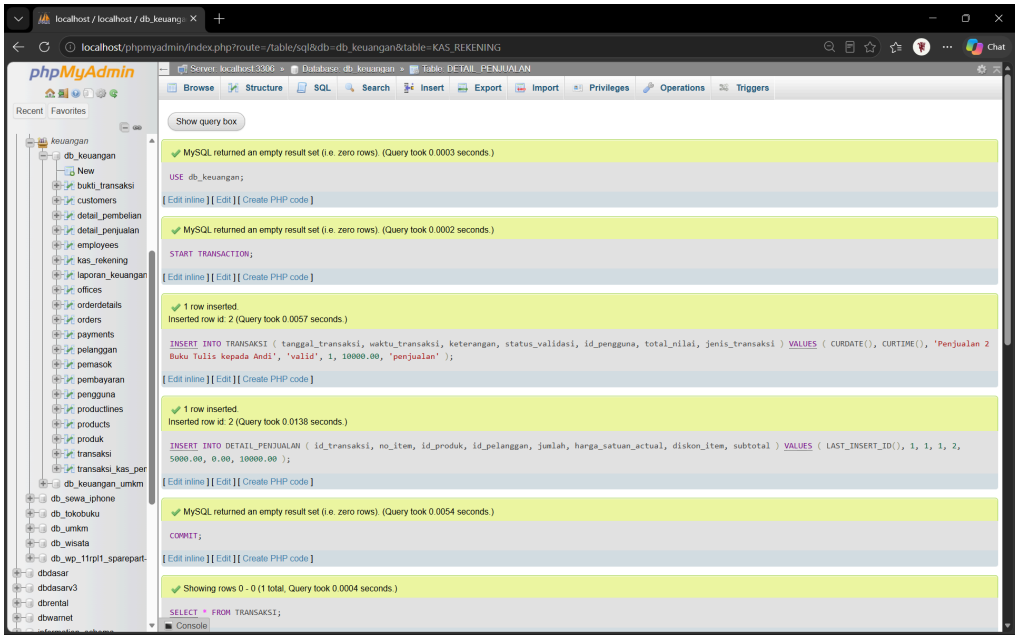
Foreign Key (FK) merupakan kolom yang merujuk pada PK tabel lain dan berfungsi menjaga integritas referensial (referential integrity). FK memastikan bahwa nilai yang dimasukkan benar-benar ada pada tabel induk, sehingga mencegah terjadinya data yatim (orphan record). Selain itu, FK mengatur aksi referensial melalui klausa ON DELETE dan ON UPDATE, seperti RESTRICT yang menolak penghapusan data induk yang masih direferensikan, CASCADE yang meneruskan perubahan ke tabel anak, serta perilaku default restrict apabila klausa tidak didefinisikan. Dalam pengujian, penyisipan DETAIL_PENJUALAN dengan id_pelanggan = 999 ditolak melalui ERROR 1452 pada constraint fk_detjual_pelanggan, dan penyisipan PEMBAYARAN dengan id_kas = 999 ditolak melalui ERROR 1452 pada constraint fk_pembayaran_kas. Keduanya membuktikan bahwa FK berhasil mencegah data yang merujuk pada entitas tidak terdaftar.

Constraint lain, seperti NOT NULL, UNIQUE, dan CHECK, melengkapi kedua mekanisme di atas dengan menjaga validitas nilai pada tingkat kolom. Constraint NOT NULL memastikan kolom wajib terisi sehingga data penting tidak kosong, UNIQUE menjamin tidak ada nilai duplikat pada kolom tertentu, sedangkan CHECK membatasi nilai agar sesuai aturan bisnis yang ditetapkan. Secara keseluruhan, PK, FK, dan constraint bekerja secara berlapis untuk menjaga integritas data pada tiga aspek sekaligus, yaitu keunikan identitas baris, konsistensi relasi antar tabel, dan validitas nilai kolom, sehingga basis data tetap akurat, konsisten, dan dapat diandalkan sebagai sumber laporan keuangan UMKM.

5. Pengujian Transaksi

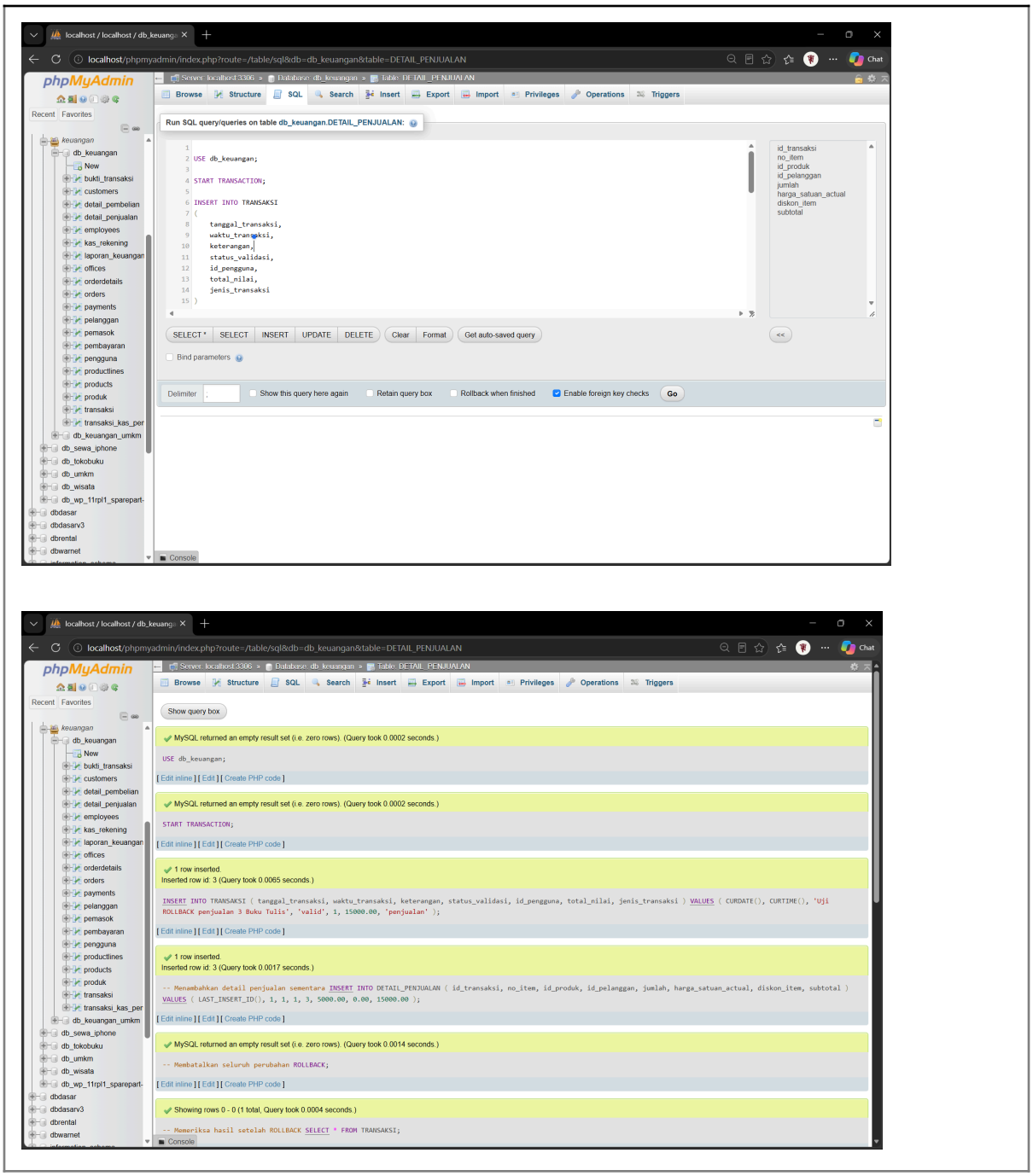
5.1 Pengujian COMMIT





Gambar 6. Bukti COMMIT

5.2 Pengujian ROLLBACK



Gambar 7. Bukti ROLLBACK

5.3 Analisis Hasil Transaksi

No	Pertanyaan	Jawaban
1	Apa perbedaan COMMIT dan ROLLBACK?	COMMIT adalah perintah untuk menyimpan semua perubahan yang dilakukan dalam sebuah transaksi secara permanen ke database. Setelah COMMIT, perubahan tidak bisa dibatalkan lagi. ROLLBACK adalah perintah untuk membatalkan semua perubahan yang dilakukan dalam sebuah transaksi, sehingga database kembali ke kondisi sebelum transaksi dimulai.
2	Mengapa operasi transaksi perlu dilakukan secara atomik?	Operasi transaksi perlu atomik karena dalam satu transaksi sering ada beberapa perintah yang harus berhasil semua atau tidak sama sekali. Kalau hanya sebagian yang berhasil, data jadi tidak konsisten. Contohnya saat menyimpan penjualan: ada perintah menyimpan pesanan, menyimpan detail pesanan, dan mengurangi stok. Kalau salah satu gagal di tengah jalan, transaksi harus dibatalkan seluruhnya agar data tidak setengah-setengah. Prinsip "semua atau tidak sama sekali" inilah yang disebut atomik (bagian dari sifat ACID).
3	Apa risiko jika transaksi penjualan hanya menyimpan tabel pesanan tetapi gagal menyimpan detailnya?	Risikonya adalah data tidak konsisten, contohnya: 1. Tabel pesanan berisi data pesanan, tetapi tidak ada detail barang yang dibeli. Akibatnya pesanan terlihat ada, tapi isinya kosong atau tidak diketahui. 2. Total harga tidak cocok dengan detail barang, sehingga laporan keuangan dan rekap penjualan menjadi salah.

		3. Stok barang tidak berkurang padahal pesanan tercatat, sehingga stok di sistem tidak sesuai kenyataan. 4. Data menjadi sulit diperbaiki karena harus dicek manual satu per satu. Karena itu, penyimpanan pesanan dan detailnya harus dilakukan dalam satu transaksi atomik: kalau detail gagal disimpan, pesanan juga harus dibatalkan (ROLLBACK).
4	Mengapa penggunaan storage engine InnoDB penting pada praktikum ini?	InnoDB penting karena hanya InnoDB yang mendukung transaksi (COMMIT, ROLLBACK) dan penguncian baris di MySQL. Storage engine lain seperti MyISAM tidak mendukung transaksi, sehingga perintah COMMIT dan ROLLBACK tidak berfungsi. Selain itu, InnoDB juga mendukung foreign key untuk menjaga relasi antar tabel (misalnya detail pesanan harus terhubung ke pesanan), serta lebih aman terhadap kerusakan data. Tanpa InnoDB, praktikum tentang transaksi tidak bisa berjalan sesuai tujuan, karena perubahan data tidak bisa dibatalkan saat terjadi kesalahan.

6. Analisis Performa

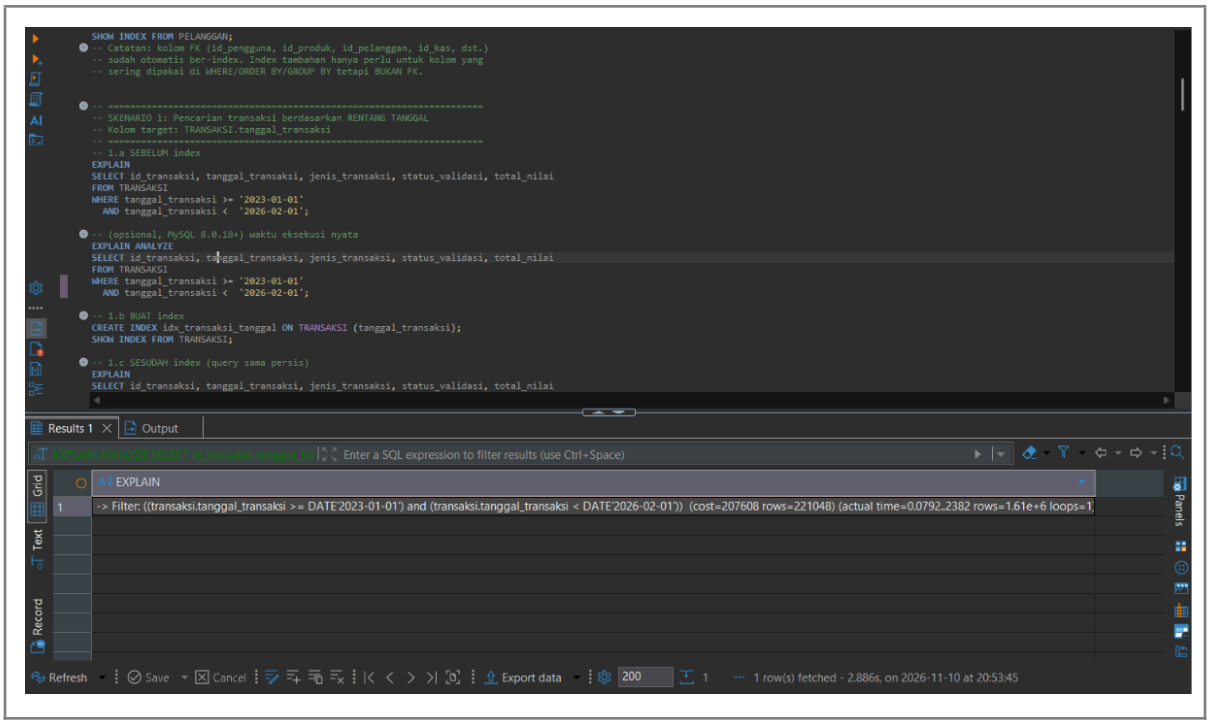
6.1 Persiapan Data

Tabel	Jumlah Baris
PENGGUNA	10000

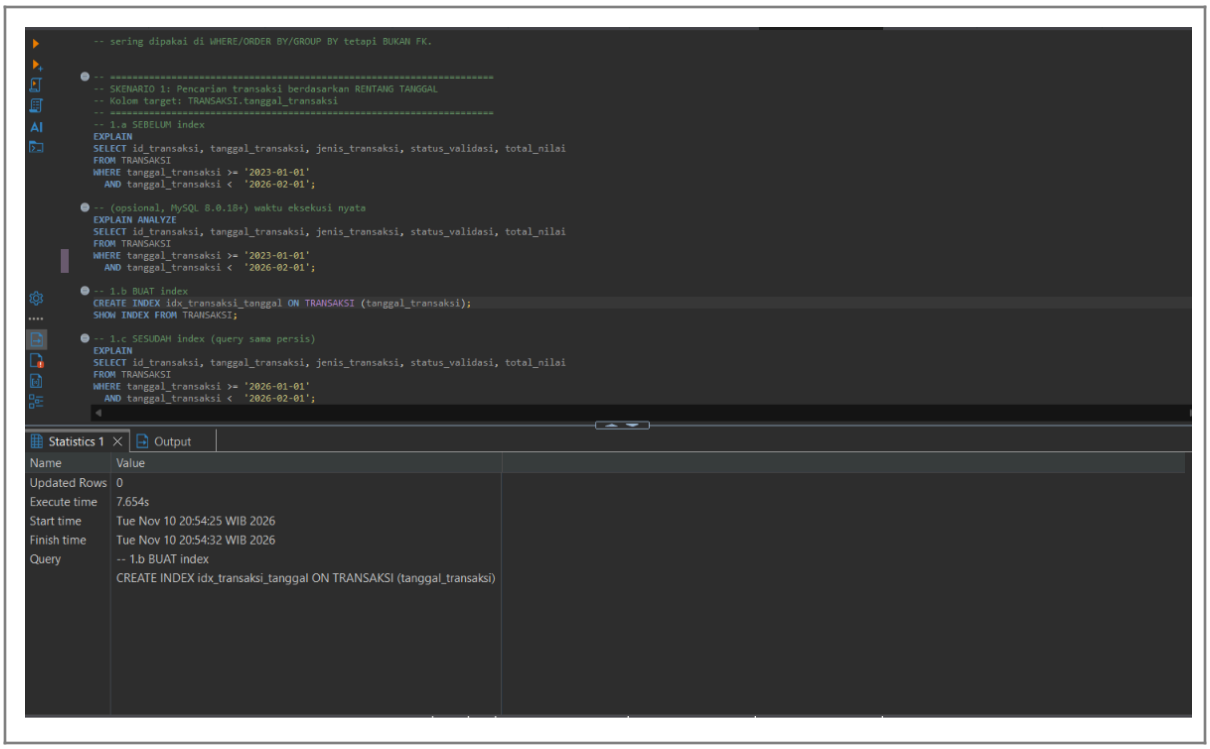
PRODUK	10000
PELANGGAN	20000
PEMASOK	10000
KAS_REKENING	10000
LAPORAN_KEUANGAN	10000
TRANSAKSI	2000000
BUKTI_TRANSAKSI	2000000
PEMBAYARAN	500000
TRANSAKSI_KAS_PEMILIK	500000
DETAIL_PENJUALAN	1000000
DETAIL_PEMBELIAN	1000000

Tabel 7. Jumlah data uji

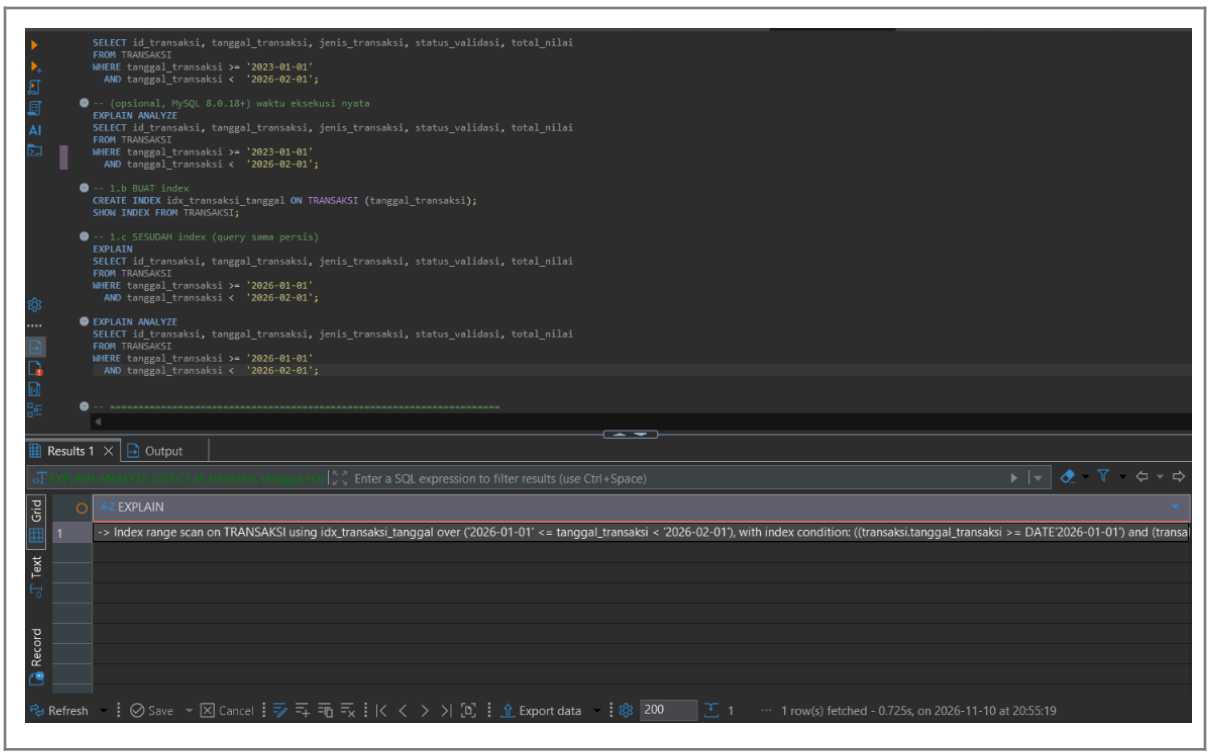
6.2 EXPLAIN Sebelum Indeks



6.3 Pembuatan Indeks



6.4 EXPLAIN Setelah Indeks



6.5 Perbandingan Hasil EXPLAIN

Kolom EXPLAIN	Sebelum Indeks	Setelah Indeks
type	ALL	range
possible_keys	idx_transaksi_tanggal	idx_transaksi_tanggal
key	null	idx_transaksi_tanggal
rows	1989831	86346
Extra	Using where	Using index condition; Using MRR
Waktu eksekusi (opsional)	2.886s	0.725s

Tabel 8. Perbandingan EXPLAIN sebelum dan sesudah indeks

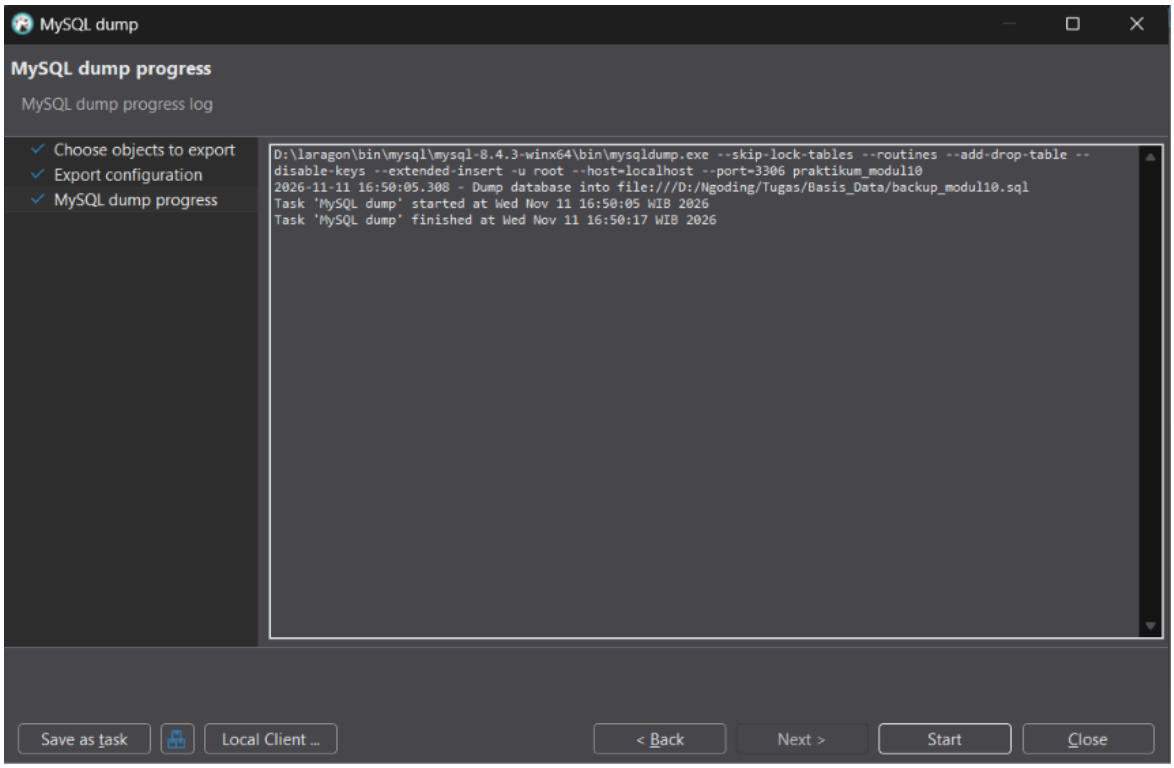
6.6 Analisis Penggunaan Indeks

No	Pertanyaan	Jawaban
1	Apa perbedaan query sebelum dan sesudah penambahan indeks?	Query menjadi lebih cepat dan rows menjadi lebih sedikit
2	Apa fungsi kolom key pada hasil EXPLAIN?	Kolom key pada hasil EXPLAIN menunjukkan index yang benar-benar dipakai oleh MySQL untuk menjalankan query
3	Apakah indeks selalu digunakan oleh optimizer?	Tidak, indeks tidak selalu digunakan oleh optimizer. Optimizer memilih berdasarkan estimasi biaya, sehingga indeks bisa diabaikan jika table scan dinilai lebih murah.
4	Mengapa indeks yang terlalu banyak dapat mengurangi performa INSERT, UPDATE, dan DELETE?	Karena setiap kali data diubah, semua indeks yang terkait juga harus ikut diperbarui. Semakin banyak indeks, semakin banyak kerja tambahan (baca-tulis disk, CPU, log, lock), sehingga operasi jadi lebih lambat.
5	Pada situasi apa indeks perlu ditambahkan?	Saat kolom sering dipakai untuk filter (WHERE), join, atau pengurutan (ORDER BY) dan datanya cukup selektif; saat

		kolom jadi foreign key; untuk pencarian rentang atau aturan unik; dan setelah dipastikan lewat query plan bahwa full scan memang jadi penghambat.

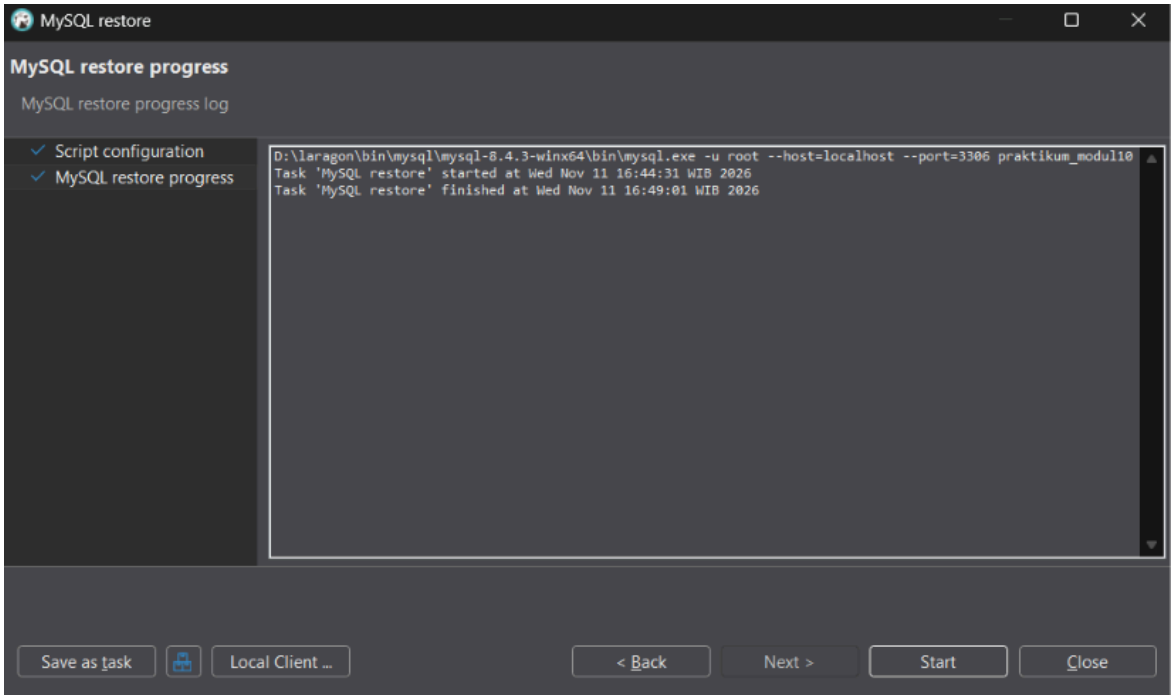
7. Pemeliharaan Basis Data

7.1 Backup



Gambar 8. Bukti backup

7.2 Restore



Gambar 9. Bukti restore

7.3 Validasi Hasil Restore

Tabel	Jumlah Baris Sebelum Backup	Jumlah Baris Setelah Restore
PENGGUNA	10000	10000
PRODUK	10000	10000
PELANGGAN	20000	20000
PEMASOK	10000	10000
KAS_REKENING	10000	10000
LAPORAN_KEUANGAN	10000	10000
TRANSAKSI	2000000	2000000
BUKTI_TRANSAKSI	2000000	2000000
PEMBAYARAN	500000	500000
TRANSAKSI_KAS_PEMILIK	500000	500000
DETAIL_PENJUALAN	1000000	1000000

Tabel	Jumlah Baris Sebelum Backup	Jumlah Baris Setelah Restore
PENGGUNA	10000	10000
PRODUK	10000	10000
PELANGGAN	20000	20000
PEMASOK	10000	10000
DETAIL_PEMBELIAN	1000000	1000000

Tabel 9. Validasi hasil restore

8. Kesimpulan

Praktikum ini menegaskan bahwa pemilihan DBMS dan alat pengelolaan yang tepat merupakan langkah awal krusial dalam pengembangan sistem informasi keuangan UMKM. Penggunaan MySQL dengan storage engine InnoDB yang dikelola melalui DBeaver terbukti menyediakan fondasi teknis yang memadai, karena mendukung fitur transaksi, integritas referensial, serta manajemen skema yang diperlukan untuk menangani data keuangan secara akurat dan efisien tanpa membebani anggaran proyek.

Rancangan basis data yang menerapkan constraint ketat dan prinsip ACID berhasil menjamin integritas data secara menyeluruh pada skala operasional nyata. Pengujian membuktikan bahwa mekanisme Primary Key, Foreign Key, dan transaksi atomik mampu mencegah inkonsistensi data, memastikan setiap entri laporan keuangan valid, serta melindungi sistem dari anomali akibat kegagalan proses di tengah eksekusi transaksi yang kompleks.

Optimasi performa melalui penerapan indeks memberikan dampak signifikan terhadap efisiensi sistem saat menangani volume data berskala besar. Analisis query plan menunjukkan bahwa indeks yang ditempatkan secara strategis pada kolom pencarian utama mampu mengubah metode akses data dari pemindaian penuh menjadi pemindaian terarah, sehingga memangkas waktu respons secara drastis tanpa mengorbankan akurasi hasil pengolahan data keuangan.

Pemeliharaan berkala dan pengamanan akses merupakan pilar penopang keberlanjutan sistem yang tidak dapat dipisahkan dari aspek teknis perancangan. Prosedur backup-restore yang tervalidasi berfungsi sebagai jaring pengaman terhadap kehilangan data, sementara pembatasan hak akses melindungi sensitivitas informasi finansial,

sehingga keseluruhan praktikum ini menghasilkan sistem basis data yang andal, aman, dan siap mendukung operasional bisnis secara berkelanjutan.

9. Pembagian Kontribusi Anggota

Skor kontribusi | sangat kurang (1-5) sangat baik |

Nama	Bagian yang Dikerjakan	Skor Kontribusi
Bramanta Setya B.	Tahap 3 dan Tahap 5	-
Edgar Rizki Putra N.	Tahap 6 dan Tahap 7	-
Rifqi Adyasta M.	Tahap 1, Tahap 2, dan Tahap 4	-

Tabel 10. Kontribusi individu

SOAL PRE TEST

1. **Jelaskan apa yang dimaksud dengan DBMS dan apa perbedaannya dengan aplikasi pengelola basis data seperti phpMyAdmin atau MySQL Workbench!**

DBMS (Database Management System) adalah perangkat lunak yang berfungsi untuk membuat, menyimpan, mengelola, dan mengakses basis data secara terstruktur, aman, dan efisien. DBMS menjadi perantara antara data yang tersimpan di disk dengan pengguna atau aplikasi yg membutuhkannya.

phpMyAdmin atau MySQL Workbench keduanya bukanlah DBMS. Keduanya adalah aplikasi klien yang menyediakan antarmuka grafis/GUI untuk berkomunikasi dengan DBMS, dalam hal ini MySQL atau MariaDB. Data tetap disimpan dan diproses oleh server DBMS-nya, sedangkan aplikasi ini hanya mengirim perintah dan menampilkan hasilnya.

2. **Apa fungsi Primary Key dan Foreign Key dalam menjaga integritas data pada basis data relasional?**

Primary Key secara unik mengidentifikasi setiap baris tabel, bersifat unik dan tidak boleh NULL. Fungsinya menjamin data agar dapat mencegah duplikasi dan memastikan setiap baris memiliki identitas tunggal.

Sedangkan Foreign Key merujuk pada primary key tabel lain dan menjaga integritas data yang memastikan relasi antar tabel valid serta mencegah data yatim/ data yang tidak memiliki induk.

3. **Mengapa penggunaan indeks dapat membantu meningkatkan performa query, dan apakah semakin banyak indeks selalu membuat basis data semakin cepat? Jelaskan!**

Karena database dapat menemukan data tanpa harus memeriksa seluruh isi tabel. Namun, terlalu banyak indeks tidak selalu baik karena dapat memperlambat proses tambah, ubah, dan hapus data serta membutuhkan ruang penyimpanan lebih besar. jadi indeks sebaiknya digunakan pada kolom yang sering dipakai untuk pencarian agar performa database tetap optimal.

SOAL POST TEST

1. **Lampirkan hasil identifikasi DBMS, versi DBMS, storage engine, aplikasi pengelola, serta diagram arsitektur sistem yang digunakan kelompok Anda. Jelaskan alasan pemilihan perangkat lunak tersebut!**

DBeaver Community Edition dipilih sebagai perangkat lunak manajemen basis data untuk proyek laporan keuangan UMKM berbasis MySQL karena memiliki fitur yang lengkap, mendukung berbagai DBMS dan sistem operasi, serta menyediakan editor query, import-export, backup-restore, dan pengelolaan hak akses. Selain itu, DBeaver bersifat open source sehingga tidak memerlukan biaya lisensi.

Meskipun memiliki antarmuka yang lebih kompleks dibandingkan phpMyAdmin, berbasis desktop, dan beberapa fitur lanjutan hanya tersedia pada versi Pro, keterbatasan tersebut tidak menjadi kendala karena basis data hanya dikelola oleh tim developer. Dengan demikian, DBeaver Community Edition dinilai sesuai dengan kebutuhan teknis dan anggaran proyek dalam mendukung pengelolaan basis data MySQL.

2. **Lampirkan script SQL pembuatan tabel beserta Primary Key, Foreign Key, dan hasil pengujian CRUD. Jelaskan bagaimana constraint yang diterapkan menjaga integritas data!**

Constraint pada basis data berfungsi untuk menjaga integritas data agar data yang disimpan tetap akurat, konsisten, dan sesuai dengan aturan yang telah ditentukan. Dalam proyek laporan keuangan UMKM berbasis MySQL, constraint diterapkan untuk mencegah kesalahan input dan menjaga hubungan antar tabel.

3. **Lampirkan hasil EXPLAIN sebelum dan sesudah pembuatan indeks. Jelaskan perubahan rencana eksekusi dan pengaruhnya terhadap pertimbangan performa basis data!**

Sebelum indeks dibuat, EXPLAIN menunjukkan type = ALL, key = NULL, dan rows yang besar, artinya MySQL membaca seluruh baris tabel (full table scan). Setelah indeks dibuat, type berubah menjadi range atau ref, key terisi nama indeks, dan rows turun drastis karena MySQL hanya membaca baris yang relevan. Pengaruhnya, query SELECT dan JOIN menjadi jauh lebih cepat, terutama pada data berjuta-juta baris. Namun, setiap indeks menambah beban pada INSERT, UPDATE, dan DELETE serta memakan ruang penyimpanan, sehingga indeks hanya perlu dibuat pada kolom yang sering dicari, digabungkan, atau diurutkan, dan manfaatnya harus dibuktikan lewat hasil EXPLAIN.

4. **Lampirkan bukti backup dan restore basis data. Jelaskan mengapa pemeliharaan dan pencadangan diperlukan dalam pengelolaan DBMS!**

Pemeliharaan dan pencadangan (backup) diperlukan agar data tetap tersedia, aman, dan dapat dipulihkan jika terjadi masalah. Data bisa hilang atau rusak karena kegagalan hardware, kesalahan pengguna (misalnya salah `DELETE` atau `DROP`), bug aplikasi, serangan siber, atau bencana, sehingga backup berfungsi sebagai salinan yang dapat di-restore untuk mengembalikan sistem seperti semula, apalagi pada data keuangan yang tidak boleh hilang. Pemeliharaan, seperti memperbarui statistik tabel (`ANALYZE TABLE`), meninjau indeks, memantau ruang penyimpanan, dan mengatur hak akses, menjaga performa serta integritas

data tetap baik seiring bertambahnya data. Backup juga perlu diuji lewat restore dan validasi jumlah data, karena backup yang tidak pernah diuji belum tentu bisa dipakai saat dibutuhkan.